

GUIA DE ESTUDO E ARQUITETURA DE SISTEMAS

Sistema de Cadastro e Avaliação de Veículos (Java)

Este documento funciona como um guia de leitura conceitual e técnico para decodificar o funcionamento interno do sistema, explicando não apenas o *que* o código faz, mas *por que* ele foi escrito dessa forma.

1. O Paradigma da Orientação a Objetos (OO)

A Orientação a Objetos é uma filosofia de desenvolvimento de software que busca **modelar o mundo real dentro do computador**. Em vez de pensarmos em programas como uma lista de comandos sequenciais, pensamos neles como um ecossistema de "peças" que conversam entre si.

O Molde (Classe) vs. A Peça Física (Objeto)

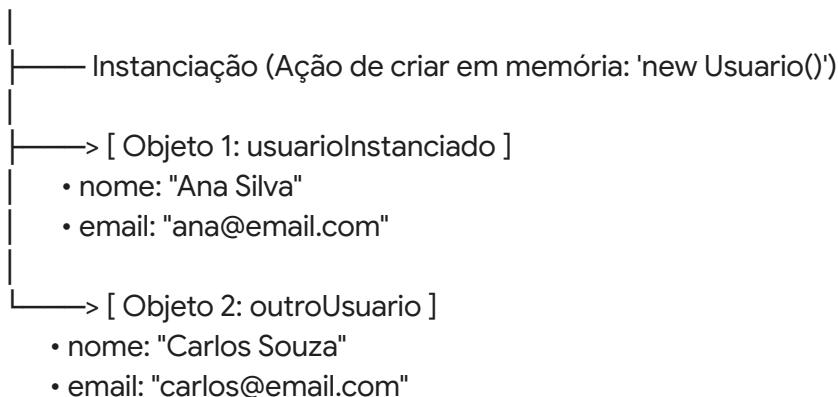
Para compreender a diferença entre Classe e Objeto, imagine uma **fôrma de bolo** e os **bolos** assados com ela:

- A **Classe** é a fôrma: ela define o formato, a altura e os detalhes que todo bolo terá, mas você não pode comer a fôrma.
- O **Objeto** é o bolo real, comestível, feito a partir daquela fôrma. Você pode fazer dez bolos idênticos; cada um deles existe de forma independente na sua mesa.

No sistema, a classe `Usuario` é a fôrma, e cada novo usuário cadastrado é um bolo único que ocupa seu próprio espaço na memória:

[Classe: `Usuario` (A Fôrma)]

- Atributos: nome, email, senha



Encapsulamento: A analogia da "Fita Isolante" e do "Painel de Controle"

Em eletrônica, fios desencapados expõem o circuito a curtos-circuitos. Na programação, permitir que qualquer parte do sistema altere diretamente as variáveis de um objeto pode gerar comportamentos inesperados.

O **Encapsulamento** resolve isso ao "isolar" os dados internos usando a palavra-chave `private`.

- **Atributos `private`:** Os dados do objeto ficam trancados dentro de uma caixa blindada.
- **Getters e Setters (A Interface Pública):** São como os botões e visores de um micro-ondas. Você não mexe na fiação interna do aparelho para aquecer a comida; você interage apenas com o painel de controle externo (get para ler a informação, set para atualizar a informação com segurança).

2. Anatomia e Estrutura das Entidades

Vamos analisar linha a linha o comportamento das peças fundamentais do nosso código.

A) Entidade Usuario — Compreendendo o Construtor e o `this`

O construtor é um bloco de código especial com o mesmo nome da classe, acionado apenas quando damos o comando `new` para dar vida a um objeto.

```
public Usuario(String nome, String email, String senha) {  
    this.nome = nome;  
    this.email = email;  
    this.senha = senha;  
}
```

- **Para que serve o `this`?**

Repare que temos duas variáveis com o nome `nome` nesse escopo: o atributo global do objeto e o parâmetro temporário recebido entre os parênteses.

Para o computador não se confundir, a palavra `this` funciona como um "dedo apontado" para si mesmo:

- `this.nome` significa: *"Este atributo aqui de dentro do meu próprio objeto".*
- `= nome` significa: *"Recebe o valor que veio lá de fora, do teclado do usuário".*

B) Entidade Veiculo — Relacionamentos e Precisão Monetária

A classe `Veiculo` lida com o vínculo entre objetos e com a representação financeira de valores de mercado.

1. Chave Estrangeira em Memória

Para associar um veículo ao seu proprietário, não guardamos o objeto inteiro do `Usuario` dentro

do carro. Guardamos apenas o seu identificador único: o emailUsuario. Isso funciona como um crachá de identificação, permitindo consultar na lista geral quem é o dono daquele veículo sem duplicar informações no computador.

2. Precisão Financeira: Por que BigDecimal é obrigatório?

Computadores usam o sistema binário (base 2) para fazer cálculos. Números como 0.1 ou 0.2 não possuem uma representação exata em binário, gerando dízimas periódicas internas no computador (da mesma forma que $\frac{1}{3}$ em decimal resulta no número infinito 0.333333...).

Se usarmos os tipos comuns double ou float para lidar com dinheiro, pequenas frações de centavos serão perdidas ou criadas em cálculos sucessivos:

Operação Matemática comum: $0.1 + 0.2 = 0.30000000000000004$

O tipo BigDecimal trata os números como texto estruturado internamente, garantindo precisão matemática decimal exata, ideal para avaliações financeiras e transações comerciais.

3. Fluxo de Controle e Gerenciamento de Memória

Onde os dados moram? (Memória RAM vs. Disco)

Este protótipo de estudo utiliza a **Memória RAM** para armazenar as listas de usuários e veículos.

- **Memória RAM:** É como a superfície de uma mesa de estudos de trabalho rápido. É extremamente ágil colocar papéis ali para ler e escrever, mas quando você sai da mesa (desliga o programa), a mesa é limpa e tudo se perde.
- **Disco Rígido (SSD/HD/Banco de Dados):** Seria como um armário de arquivos físico. Mais lento para acessar, mas mantém tudo guardado após fechar a sala. Por ser um protótipo focado nos conceitos estruturais da linguagem, a RAM atende perfeitamente à nossa necessidade de teste ágil.

ArrayList: A gaveta que estica

Vetores (Arrays) tradicionais em Java nascem com tamanho fixo. Se você criar um array de 5 posições, o sexto elemento quebrará o sistema.

O ArrayList<T> funciona como uma **gaveta inteligente com fundo falso**. Sempre que ela fica cheia, o Java silenciosamente cria uma gaveta maior no fundo, copia as coisas antigas para lá e abre espaço para novos itens, crescendo conforme a necessidade do cadastro.

O Conceito de Membros Estáticos (static)

Pense no modificador static como um **recurso compartilhado por toda a vizinhança**.

- Sem a palavra `static`, cada objeto criado precisa ter sua própria cópia daquela variável.
- Com a palavra `static`, a variável pertence à classe como um todo. Todas as partes do sistema leem e alteram a **mesma única lista**, de forma centralizada e sem a necessidade de instanciar a classe `Main` repetidas vezes.

4. Engenharia de Resiliência: Blindando o Terminal

Erros de digitação dos usuários são inevitáveis. Uma aplicação robusta não pode simplesmente "travar" e fechar na cara do operador se ele digitar uma letra onde deveria digitar um preço.

A Rede de Proteção: `try-catch`

O mecanismo do `try-catch` funciona exatamente como uma rede de trapézio de circo:

```
try {
    // O trapezista tenta fazer o salto arriscado:
    return Integer.parseInt(entrada);
} catch (NumberFormatException e) {
    // Se ele cair (digitar letras), a rede o segura com segurança:
    System.out.println("Erro: digite um número inteiro válido.");
}
```

Sem essa estrutura, se o usuário digitasse "dez", o método `parseInt` causaria um travamento completo da Máquina Virtual Java (JVM). Com o `try-catch`, nós capturamos a queda, avisamos o usuário do erro de digitação e permitimos que o programa continue rodando.

O Filtro de Linha: Expressões Regulares (Regex)

Antes mesmo de enviar o dado para conversão, o sistema usa uma máscara de validação conhecida como **Regex** `(\d+(\.\d+)?)` para verificar a conformidade dos caracteres:

<code>\d+</code>	<code>(\. \d+) ?</code>
▼	▼
Um ou mais números de 0-9	Um ponto decimal é opcional (pode ou não existir)

Isso impede que tentativas de conversão malsucedidas sequer aconteçam, agilizando o processamento e garantindo que apenas formatos monetários corretos (ex: 15000 ou 23500.50) avancem no fluxo.

5. Algoritmos de Busca: Como o sistema localiza

dados?

Para encontrar todos os veículos pertencentes a um determinado usuário, o sistema utiliza o algoritmo de **Busca Sequencial**:

```
private static ArrayList<Veiculo> getVeiculosDoUsuario(String emailUsuario) {  
    ArrayList<Veiculo> resultado = new ArrayList<>();  
    for (Veiculo v : veiculos) {  
        if (v.getEmailUsuario().equalsIgnoreCase(emailUsuario)) {  
            resultado.add(v);  
        }  
    }  
    return resultado;  
}
```

Imagine que você está em uma biblioteca procurando todos os livros escritos por um autor específico:

1. Você cria uma pilha vazia chamada resultado.
2. Você vai até a estante global de livros (veiculos).
3. Pega livro por livro, do primeiro ao último (for-each).
4. Olha para a etiqueta do autor no livro (getEmailUsuario()).
5. Se for o autor que você quer (ignorando letras maiúsculas ou minúsculas por meio de equalsIgnoreCase), você coloca uma cópia na sua pilha de resultado.
6. No fim do corredor, você entrega essa nova pilha montada.